

Trabalho Prático 2

Ítalo Dell’Areti

Raquel Gonçalves Rosa

Universidade Federal de Minas Gerais (UFMG)

Belo Horizonte – MG – Brasil

italodellareti@ufmg.br, raquelgr@ufmg.br

1. Introdução

Este trabalho tem como objetivo implementar e analisar um simulador de memória virtual que reproduza o comportamento de diferentes algoritmos de substituição de páginas e estruturas de tabela de páginas. O simulador processa sequências reais de acessos à memória de programas diversos, permitindo a avaliação comparativa de estratégias de gerenciamento de memória em cenários controlados.

2. Motivação e Escolha dos Algoritmos

2.1. Algoritmos de Substituição de Páginas Seleccionados

- **Random:** Base para comparação, representando o pior caso de desempenho possível sem nenhuma estratégia inteligente.
- **LRU (Least Recently Used):** Algoritmo clássico baseado na localidade temporal, assume que páginas acessadas recentemente têm maior probabilidade de serem acessadas novamente. Possui um bom desempenho.
- **LFU (Least Frequently Used):** Prioriza páginas mais frequentemente acessadas, eficaz em padrões repetitivos.
- **Clock (Segunda Chance):** Algoritmo escolhido porque combina simplicidade com eficiência, usando um ponteiro circular e bit de referência. Páginas referenciadas recebem segunda chance antes da substituição. Oferece boa aproximação do LRU com menor overhead computacional.

2.2. Estruturas de Tabela de Páginas

- **Tabela Densa:** Acesso direto $O(1)$, possui maior consumo de memória, mas com acesso muito rápido.
- **Hierárquica 2 Níveis:** Ocupa menos espaço de memória em comparação com a tabela densa e faz balanceamento entre memória e desempenho.
- **Hierárquica 3 Níveis:** Maior economia de espaço para endereçamento muito esparsos. Possui menor consumo de memória, mas com maior overhead de acesso.
- **Tabela Invertida:** Espaço proporcional ao número de quadros físicos, não ao espaço de endereçamento virtual. Consumo de memória fixo proporcional aos quadros físicos, busca mais complexa.

3. Resumo do projeto

3.1. Estrutura Geral do Código

- **simulador.c:** Módulo principal, responsável pelo controle do fluxo de execução, *parsing* de argumentos da linha de comando, e coordenação entre todos os módulos através de ponteiros de função. Possui também a função *handle_memory_access()* que processa cada acesso à memória.
- **page_table.c:** Módulo que implementa as quatro estruturas de tabela de páginas com suas respectivas funções de inicialização, busca, atualização e limpeza.
- **replacement.c:** Módulo que implementa os quatro algoritmos de substituição. Cada função retorna o índice do quadro a ser substituído quando a memória está cheia.
- **utils.c:** Módulo que contém duas funcionalidades adicionais:
 - *benchmark_page_tables()* para medir a performance das estruturas de tabela.
 - *create_test_files()* para gerar arquivos de teste com padrões definidos.
- **gerar_graficos.py:** Script Python para gerar gráficos comparativos e relatórios a partir dos dados coletados durante os experimentos.

3.2. Estruturas de Dados Principais

- **Frame:** representa um frame físico na memória

```
typedef struct {
    unsigned page_number;    // Número da página virtual armazenada
    unsigned long last_used; // Timestamp do último acesso (para LRU)
    unsigned access_count;   // Contador de acessos (para LFU)
    bool valid;              // Se este frame contém uma página válida
    bool dirty;              // Se esta página foi modificada
    bool referenced;         // Bit de referência (para Clock)
} Frame;
```

- **Tabela de Páginas Densa**

```
typedef struct {
    int *entries;            // Array direto: índice = página, valor = frame
    unsigned size;           // Número total de entradas ( $2^{(32-s\_bits)}$ )
} DensePageTable;
```

- **Tabela Hierárquica de 2 Níveis**

```
typedef struct {
    int **directory;         // Diretório apontando para tabelas de segundo nível
    unsigned dir_size;       // Número de entradas no diretório
    unsigned table_size;     // Tamanho de cada tabela de segundo nível
} TwoLevelPageTable;
```

- **Tabela Hierárquica de 3 Níveis**

```
typedef struct {
    int ***directory;        // Diretório de três níveis de indireção
    unsigned dir_size;       // Tamanho do diretório superior
    unsigned mid_size;       // Tamanho dos diretórios médios
    unsigned table_size;     // Tamanho das tabelas finais
} ThreeLevelPageTable;
```

- **Tabela Invertida**

```
typedef struct {  
    unsigned *virtual_pages; // Mapeia frame físico → página virtual  
    unsigned size;           // Número de frames (= total_frames)  
    bool *valid;             // Indica se cada entrada é válida  
} InvertedPageTable;
```

4. Decisões de Projeto

4.1 Resolução de Ambiguidades da Especificação

- **Inicialização da Memória:**
 - Todos os frames são inicializados como inválidos no início da simulação
 - Frames livres são preenchidos primeiro antes de invocar algoritmos de substituição
 - Metadados de cada frame (timestamp, contador de acesso, bit de referência) são zerados na inicialização
- **Tratamento de Páginas Sujas**
 - Páginas são marcadas como sujas apenas quando modificadas
 - Páginas sujas são escritas de volta ao disco somente quando substituídas
 - Páginas sujas remanescentes ao final da simulação não são contabilizadas como escritas
- **Implementação dos Algoritmos:**
 - **Random:** Utiliza *random() % total_frames* para seleção aleatória, com *seed* inicializado via *srandom(time(NULL))*.
 - **LRU:** Utiliza timestamp global incrementado a cada acesso e armazenado. O frame com menor timestamp é escolhido para substituição.
 - **LFU:** Mantém contador em cada frame, incrementado a cada acerto. O frame com menor contador é escolhido para substituição.
 - **Clock:** Implementa ponteiro circular estático que percorre os frames. Frames já referenciados recebem segunda chance e têm o bit limpo.
- **Parsing de Argumentos:**
 - Implementamos flexibilidade para 5º e 6º argumentos opcionais
 - Se tiver apenas 5 argumentos: valores 0-3 são interpretados como tipo de tabela, valores > 3 como nível de debug
 - Se tiver 6 argumentos: 5º é tipo de tabela, 6º é nível de debug

- Modos especiais: debug=8 (*benchmark*), debug=9 (gerar arquivos de teste)

4.2 Decisões de Implementação

4.2.1. Estruturas de Tabela

- **Tabela Densa:** Array único com -1 para marcar não utilização.
- **Hierárquicas:** Alocação sob demanda com divisão de *bits* equilibrada.
- **Invertida:** Busca linear com array de validade paralelo

4.2.2. Otimizações

- Ponteiros de função para evitar switch/case em cada acesso
- Pré-cálculo de *bits de offset* na inicialização

5. Análise de Resultados

5.1. Análise do Impacto do Tamanho da Memória

A figura 1 (página x) apresenta o comportamento dos algoritmos conforme o tamanho da memória varia de 128KB a 2048KB, mantendo a página fixa em 4KB.

No gráfico *Taxa de Page Faults vs Tamanho da Memória*, observamos curvas exponenciais decrescentes para todos os algoritmos, mas com características bem diferentes. A curva mais drástica pertence ao LFU, que inicia em aproximadamente 60% de *page faults* com 128KB e despenca para cerca de 2% com 2048KB. Esta queda abrupta sugere que o LFU sofre severamente quando o *working set* não cabe completamente na memória, mas torna-se aceitável quando há mais recursos disponíveis. Em contraste, LRU e Clock apresentam curvas praticamente sobrepostas, iniciando em torno de 8-9% com 128KB e diminuindo suavemente para 1% com 2048KB. Esta sobreposição visual confirma que o algoritmo de Clock aproxima efetivamente o comportamento do LRU.

O gráfico de *Páginas Lidas vs Tamanho da Memória* mostra ainda mais estas disparidades. O LFU apresenta picos que decrescem exponencialmente, atingindo valores próximos a 600.000 páginas lidas com pouca memória. LRU e Clock mantêm curvas baixas e próximas, enquanto Random se posiciona consistentemente entre os algoritmos eficientes e o LFU problemático.

No gráfico de *Páginas Escritas vs Tamanho da Memória*, o padrão se repete com o LFU mostrando valores extremos que sugerem *thrashing* severo - o sistema gasta mais recursos movendo dados entre memória e disco do que executando trabalho útil. O gráfico *Comparação das Taxas de Page Faults* mostra claramente estas disparidades em diferentes pontos de memória, mostrando o LFU como outlier consistente.

Um ponto crítico observado é a convergência em 2048KB, onde todos os algoritmos apresentam performance similar, isso indica que o *working set* dos programas testados cabe confortavelmente nesta quantidade de memória, tornando a escolha do algoritmo menos relevante quando há muita disponibilidade de recursos.

5.2. Análise do Impacto do Tamanho da Página

A **figura 2 (página x)** apresenta o comportamento dos algoritmos conforme o tamanho das páginas varia de 2KB a 64KB, mantendo a memória fixa em 512KB.

No gráfico *Taxa de Page Faults vs Tamanho da Página*, observamos que LRU, Clock e Random apresentam aumento gradual na taxa de *page faults* conforme as páginas crescem. A explicação está na matemática por trás do experimento: com memória fixa de 512KB, páginas de 2KB resultam em 256 *frames* disponíveis, enquanto páginas de 64KB permitem apenas 8 *frames*. Esta redução drástica no número de *frames* limita a capacidade dos algoritmos de manter *working sets* adequados na memória.

O LFU mostra um comportamento completamente diferente e problemático. A partir de páginas de 8KB, sua taxa de *page faults* cresce exponencialmente, atingindo picos superiores a 60% com páginas de 16-32KB. Este comportamento indica que o algoritmo é particularmente sensível à granularidade de gerenciamento. É possível uma ligeira melhora com 64KB, possivelmente porque com apenas 8 *frames* disponíveis, há menos oportunidades para decisões ruins baseadas em frequência histórica.

O gráfico *Páginas Lidas vs Tamanho da Página* mostra ainda mais estas diferenças, o LFU segue com crescimento exponencial que atinge valores próximos a 650.000 páginas lidas com páginas grandes. Enquanto os outros algoritmos mantêm crescimento controlado e próximo.

No gráfico *Páginas Escritas vs Tamanho da Página*, o LFU novamente apresenta valores extremos, confirmando sua ineficiência crescente. O *heatmap Taxa de Page Faults por Algoritmo e Tamanho da Página* oferece uma visualização clara através de cores: LRU, Random e Clock mantêm tons claros, ou seja, baixas taxas em todas as configurações, enquanto o LFU progride para tons escuros, especialmente na faixa crítica de 16-32KB.

5.3. Análise por Workload Específico

A **figura 3 (página x)** mostra como diferentes tipos de programa afetam o desempenho relativo dos algoritmos, demonstrando que características específicas do *workload* podem favorecer ou prejudicar determinadas estratégias de substituição.

No gráfico *Taxa de Page Faults por Workload*, observamos variações significativas entre os diferentes programas. Para o *workload* "compilador", LRU e Clock apresentam desempenho muito próximo (aproximadamente 8-9%), Random mostra degradação moderada (cerca de 11%), enquanto LFU falha catastroficamente com mais de 60%. Este padrão reflete as características de compiladores que fazem uso intensivo de estruturas complexas com alta localidade temporal.

O *workload* "matriz" apresenta o melhor cenário geral, onde LRU atinge cerca de 5% de *page faults*, Clock mantém proximidade competitiva, e até Random apresenta valores aceitáveis. Este comportamento é consistente com operações matriciais que apresentam padrões sequenciais bem definidos.

O "compressor" representa um caso excepcional onde LRU, Clock e Random alcançam performance quase perfeita (abaixo de 1%), não sendo nem muito visível no gráfico. Mesmo neste cenário favorável, o LFU falha com aproximadamente 40%, demonstrando limitações fundamentais da abordagem por frequência. O "simulador" exibe o pior caso para LFU, ultrapassando 70% de *page faults*, enquanto os outros se mantêm abaixo de 10%.

O gráfico *Páginas Lidas por Workload* e *Páginas Escritas por Workload* reforçam estes padrões, mostrando o LFU com valores consistentemente elevados em todos os *workloads*. O *heatmap Taxa de Page Faults por Algoritmo e Workload* sintetiza visualmente estas diferenças, apresentando cores azuis para LRU, Random e Clock enquanto apresenta cores vermelhas para LFU em todos os programas testados.

5.4. Análise Comparativa Geral

A **figura 4 (página x)** consolida todos os experimentos realizados, oferecendo uma visão geral da eficácia relativa dos algoritmos através de múltiplas métricas e perspectivas.

O gráfico *Taxa Média de Page Faults por Algoritmo* estabelece a hierarquia final: LRU lidera com 5.4%, Clock segue próximo com 5.7%, Random ocupa posição intermediária com 7.2%, e LFU aparece como *outlier* extremo com 37.7%. A proximidade entre LRU e Clock é visualmente evidente, confirmando que Clock oferece eficiência quase idêntica.

No gráfico *Eficiência Relativa dos Algoritmos*, onde LRU serve como referência (1.0), observamos que Clock apresenta degradação mínima (1.05), Random mostra degradação moderada (1.32), enquanto LFU exibe uma alta degradação (6.94). A linha tracejada vermelha em 1.0 enfatiza quão próximo Clock permanece do ideal.

A *Comparação Multidimensional Normalizada* considera simultaneamente *page faults*, páginas lidas e páginas escritas invertidas em escala 0-100. LRU e Clock apresentam pontuações altas e similares em todas as dimensões, Random mantém performance mediana, enquanto LFU apresenta pontuações baixas, especialmente em *page faults*.

O *Ranking dos Algoritmos* oferece a visualização mais direta da hierarquia através de barras horizontais. A diferença visual entre LFU e os demais algoritmos enfatiza a magnitude da disparidade observada em todos os experimentos.

5.5. Conclusões da Análise

A análise dos gráficos mostra padrões consistentes que permitem conclusões definitivas sobre a eficácia dos algoritmos testados. LRU e Clock demonstram comportamento virtualmente idêntico em todas as condições, com Clock oferecendo cerca de 95% da eficiência do LRU. Esta proximidade sugere que Clock é uma alternativa superior quando a simplicidade de implementação é valorizada.

Random cumpre seu papel como *baseline*, mantendo posição intermediária estável em todos os cenários. Sua previsibilidade o valida como ponto de referência neutro para avaliar outros algoritmos.

LFU surge como consistentemente inadequado, apresentando falhas sistemáticas em todas as visualizações. Sua inadequação não é específica de determinados cenários, mas fundamental, sugerindo que abordagens baseadas em frequência histórica são problemáticas para *workloads* modernos.

Os pontos de convergência com alta memória e divergência em condições restritivas indicam que a escolha do algoritmo é crítica em sistemas com recursos limitados, mas menos relevante para muitos recursos de memória. Esta observação tem implicações práticas para o *design* de sistemas reais.

6. Conclusão

Os resultados obtidos confirmaram parcialmente nossas expectativas iniciais, mas mostraram dados importantes que divergiram significativamente do previsto. Como esperado, o LRU demonstrou superioridade consistente em *workloads* com localidade temporal, confirmando sua posição como referência em sistemas de produção, enquanto o Random cumpriu efetivamente seu papel como *baseline* neutro.

Os outros dois algoritmos foram mais diferentes. Primeiro, a proximidade entre LRU e Clock foi muito além do esperado - cerca 0.20% de diferença na taxa média de *page faults* o que demonstra que Clock é praticamente equivalente ao LRU em eficiência, mas com implementação significativamente mais simples. Segundo, a alta magnitude de falha do LFU foi alarmante. Com taxa média de 23% de *page faults* (mais de cinco vezes pior que LRU) e picos de 71% no simulador, o LFU falhou constantemente mesmo em cenários aparentemente favoráveis como o compressor, onde apresentou 41% contra apenas 0.20% do LRU.

A eficácia do Clock explica-se pela capacidade do bit de referência capturar a essência da localidade temporal sem necessidade de ordenação temporal completa. O mecanismo de "segunda chance" preserva páginas recentemente acessadas enquanto marca naturalmente páginas não utilizadas para substituição. A falha do LFU deriva de dois problemas fundamentais: "poluição temporal" onde páginas antigas frequentes ocupam memória desnecessariamente, e inadequação para padrões sequenciais onde baixa frequência individual não reflete importância atual. Adicionalmente, páginas novas sempre iniciam com frequência zero, sendo expulsas prematuramente mesmo quando fazem parte do *working set* ativo.

Nosso estudo apresenta limitações que devem ser consideradas: utilização de apenas quatro *workloads* específicos, foco em configurações particulares sem considerar multiprogramação, ausência de análise de overhead computacional real, e não inclusão de algoritmos híbridos ou adaptativos modernos.

Com base nos resultados, podemos concluir que para sistemas onde performance é crítica, LRU permanece como escolha ideal devido ao seu desempenho superior consistente em todos os cenários testados. Para sistemas com restrições de implementação ou onde simplicidade é valorizada, Clock oferece uma alternativa com 95% da eficiência do LRU, mas com complexidade computacional significativamente menor. O LFU deve ser evitado em sistemas de produção, baseado na evidência empírica que demonstrou sua inadequação.

Por fim, este trabalho fornece contribuições importantes ao demonstrar quantitativamente que localidade temporal é muito mais relevante que frequência histórica para prever padrões de

acesso em *workloads* modernos. Comprovamos que Clock oferece eficiência quase idêntica ao LRU, validando sua viabilidade como alternativa prática.